

LAB REPORT : 01

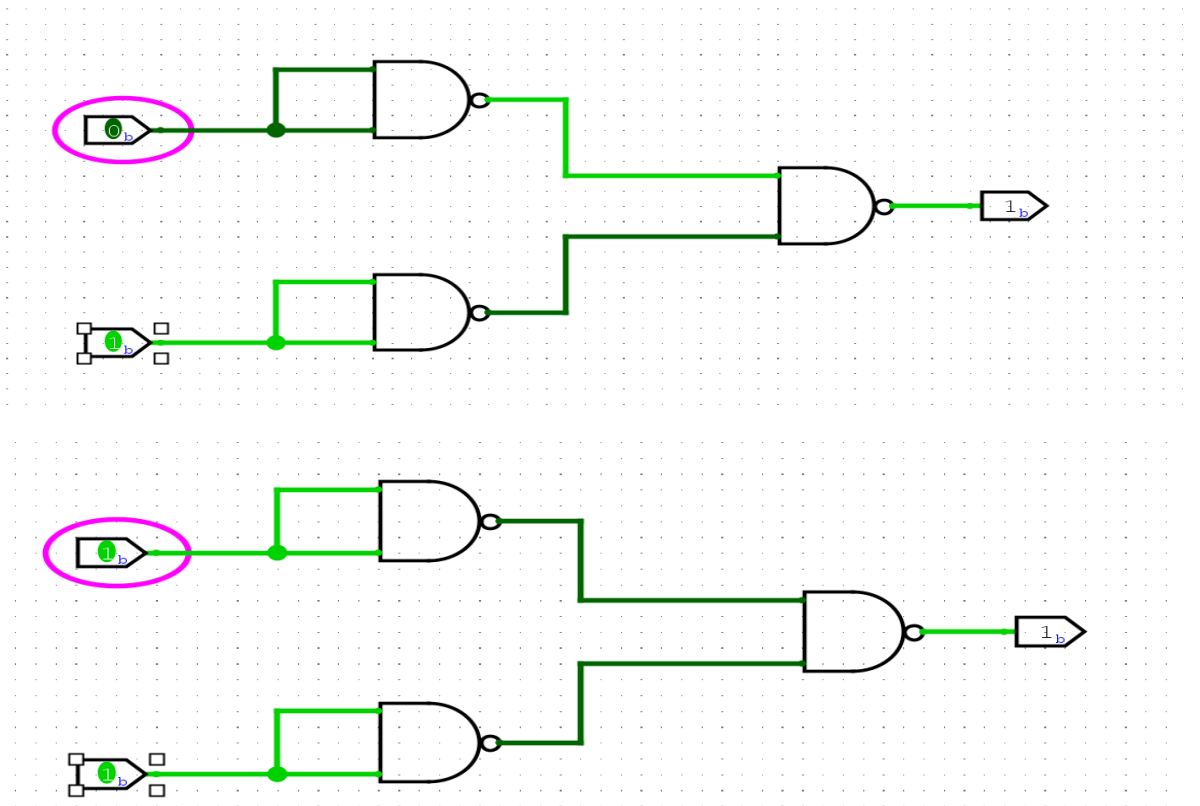
Task 1 : Implementing OR Gate Using NAND Gates

Theory:

1. An OR gate can be implemented using three NAND gates by applying De Morgan's theorem.
2. The first two NAND gates are used as NOT gates by connecting each input to both terminals.
3. This produces the complemented inputs, $\bar{A}$ and $\bar{B}$.
4. The third NAND gate combines these outputs to produce $(\bar{A} \cdot \bar{B})$, which is equivalent to $A + B$.
5. Therefore, an OR gate is realized using only NAND gates with the expression

$$Y = (\bar{A} \cdot \bar{B}) = A + B.$$

Circuit design:



Truth Table:

A	B	$\bar{A}$	$\bar{B}$	$(\bar{A} \cdot \bar{B}) = Y$
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

Discussion

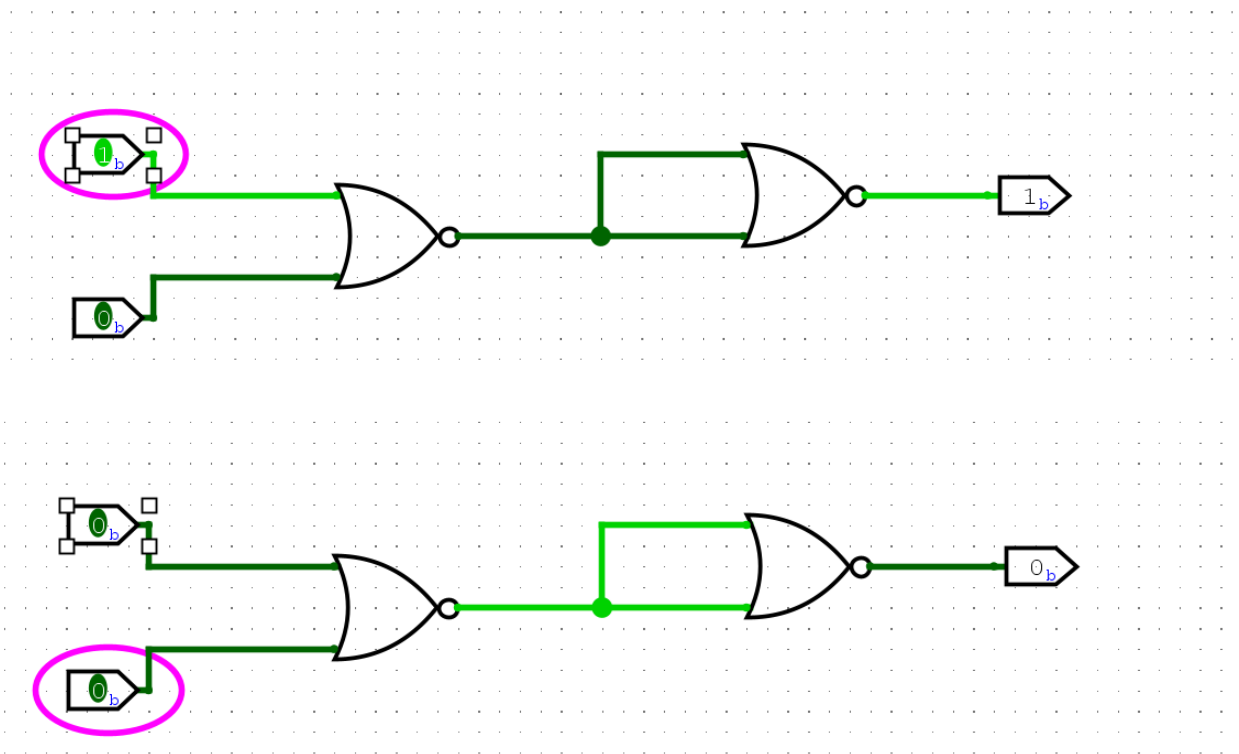
The circuit shown implements an **OR gate using three NAND gates**. The first two NAND gates have their inputs connected together, causing each to function as a **NOT gate (inverter)**. The upper NAND gate produces $\bar{A}$, while the lower NAND gate produces $\bar{B}$. These complemented outputs are then connected to the third NAND gate. The third NAND gate performs the operation $(\bar{A} \cdot \bar{B})$, which, according to **De Morgan's theorem**, is equivalent to $A + B$. Therefore, the final output behaves exactly like an OR gate. The circuit verifies that a NAND gate is a **universal gate**, capable of implementing any logic function

Task 2 : Implementing OR Gate Using NOR Gates

Theory:

1. An OR gate can be implemented using **two NOR gates**.
2. The first NOR gate performs the OR operation and produces the complemented output $(A + B)'$.
3. The second NOR gate is used as a **NOT gate** by connecting both of its inputs together.
4. It inverts $(A + B)'$ to produce $((A + B)')'$.
5. Therefore, the final output is $A + B$, which is the OR function.

Circuit Diagram:



Truth Table:

A	B	$(A + B)'$	$Y = ((A + B)')' = A + B$
0	0	1	0
0	1	0	1
1	0	0	1
1	1	0	1

Discussion

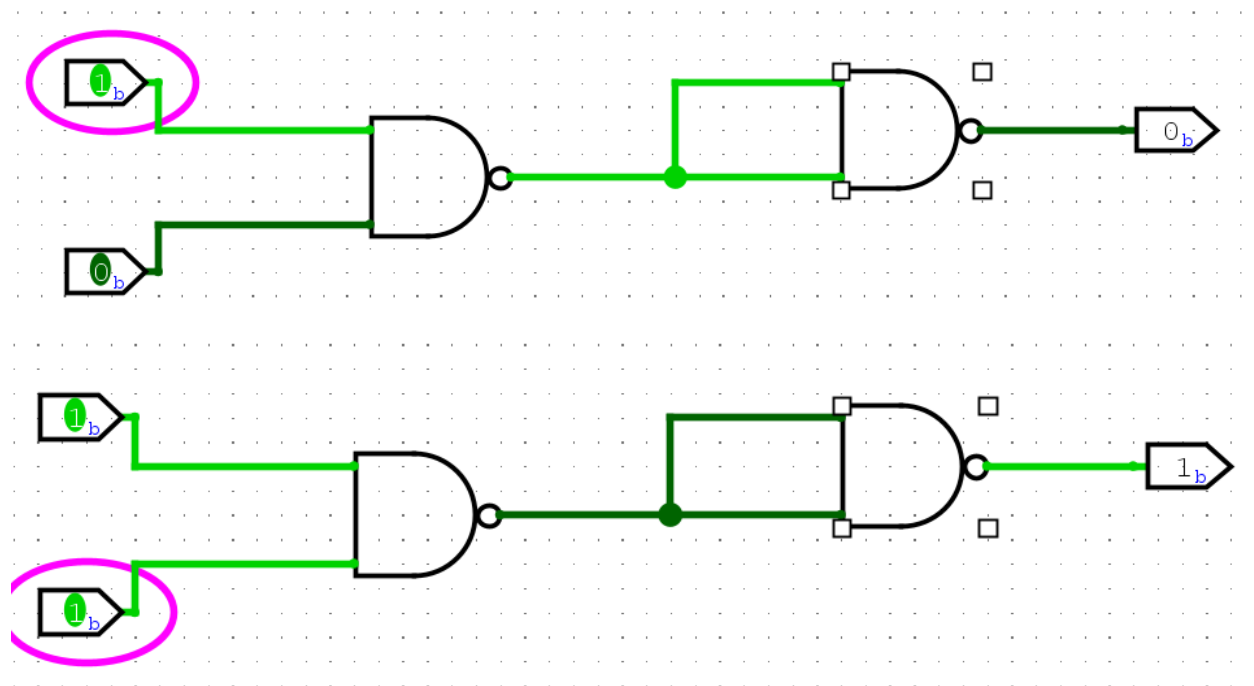
The circuit implements an OR gate using two NOR gates. The first NOR gate accepts inputs A and B and generates the complemented output $(A + B)'$. This output is connected to both inputs of the second NOR gate, allowing it to operate as an inverter. The second NOR gate complements the signal again, producing $((A + B)')'$, which simplifies to $A + B$. Hence, the circuit behaves exactly like a standard OR gate. This implementation demonstrates that the NOR gate is a universal logic gate because it can be used to realize the OR function.

Task 3: Implementing AND Gate Using NAND Gates

Theory:

1. An AND gate can be implemented using **two NAND gates**.
2. The first NAND gate performs the NAND operation and produces the output $(A \cdot B)'$.
3. The output of the first NAND gate is connected to both inputs of the second NAND gate.
4. The second NAND gate acts as a **NOT gate**, inverting $(A \cdot B)'$ to $((A \cdot B)')'$.
5. Therefore, the final output is $A \cdot B$, which is the AND function.

Circuit Diagram:



Truth Table:

A	B	$(A \cdot B)'$	$Y = ((A \cdot B)')' = A \cdot B$
0	0	1	0
0	1	1	0
1	0	1	0

A	B	$(A \cdot B)'$	$Y = ((A \cdot B)')' = A \cdot B$
1	1	0	1

Discussion

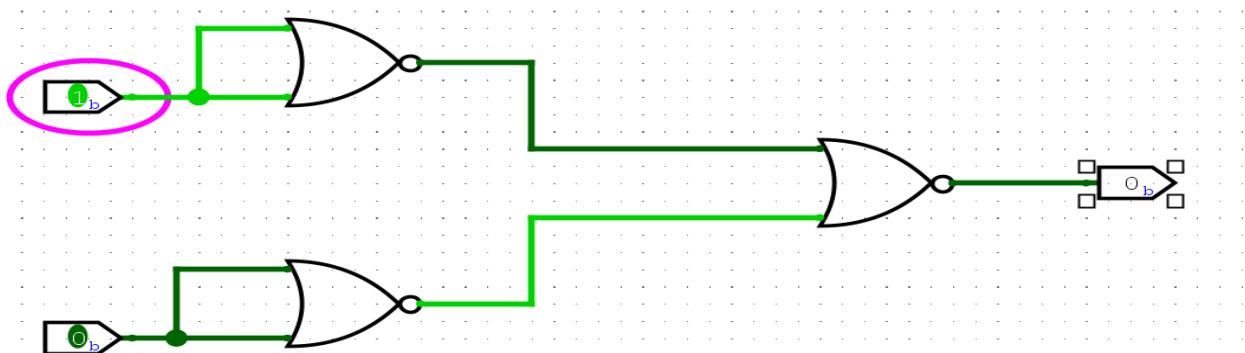
The circuit implements an AND gate using two NAND gates. The first NAND gate receives inputs A and B and generates the complemented output $(A \cdot B)'$. This output is connected to both inputs of the second NAND gate, causing it to function as an inverter. The second NAND gate complements the signal again, producing $((A \cdot B)')'$, which simplifies to $A \cdot B$. Thus, the circuit behaves exactly like a standard AND gate. This implementation demonstrates that the NAND gate is a universal logic gate, capable of realizing the AND function.

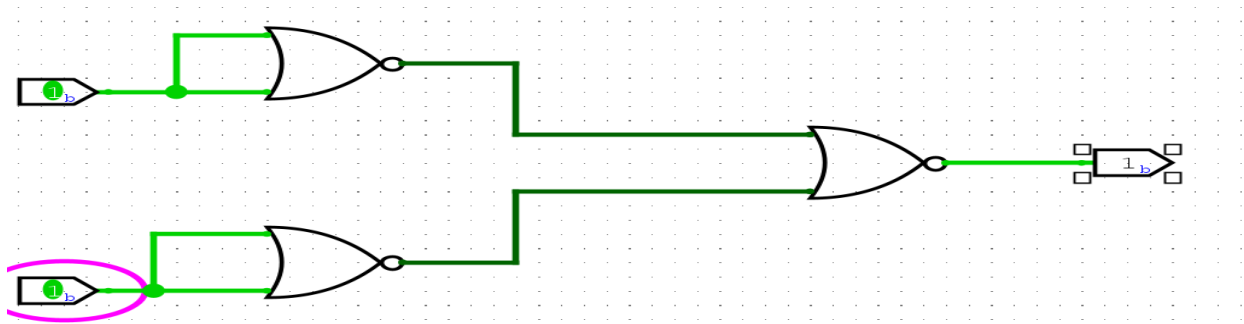
Task 4: Implementing AND Gate Using NOR Gates

Theory:

1. An AND gate can be implemented using **three NOR gates**.
2. The first two NOR gates are used as **NOT gates** by connecting each input to both terminals.
3. This produces the complemented inputs **A'** and **B'**.
4. The third NOR gate combines these complemented inputs to produce $(A' + B')'$.
5. By **De Morgan's theorem**, $(A' + B')' = A \cdot B$, which is the AND function.

Circuit Diagram:





Truth Table:

A	B	A'	B'	$Y = (A' + B')' = A \cdot B$
0	0	1	1	0
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

Discussion

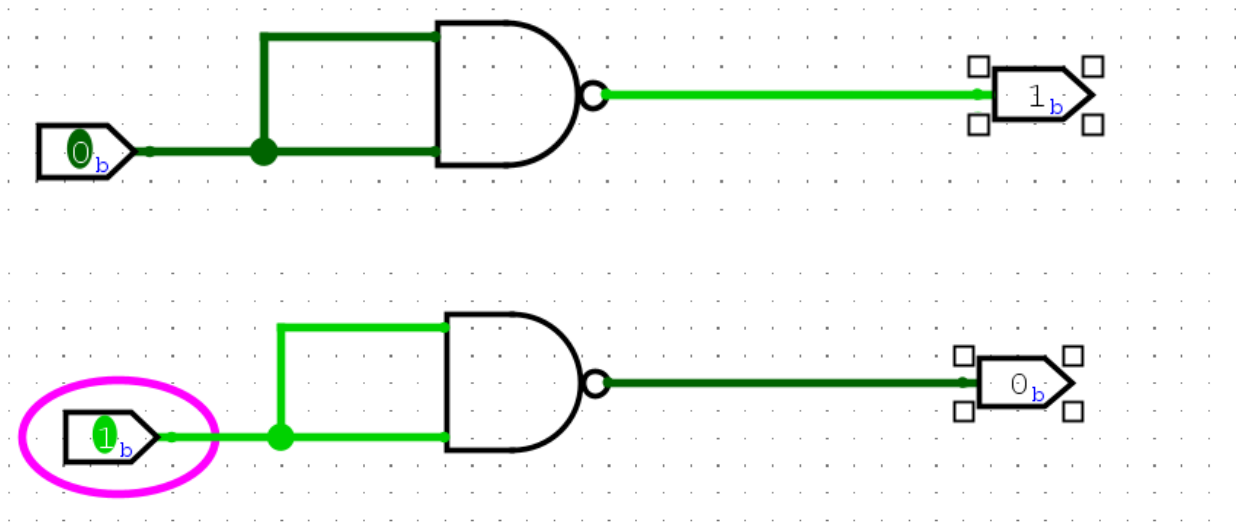
The circuit implements an AND gate using three NOR gates. The first two NOR gates act as inverters by connecting each input to both terminals, producing A' and B' . These complemented signals are then applied to the third NOR gate. The third NOR gate generates the output $(A' + B')'$, which, according to De Morgan's theorem, is equivalent to $A \cdot B$. Thus, the circuit performs the same operation as a standard AND gate. This implementation demonstrates that the NOR gate is a universal logic gate and can be used to realize the AND function.

Task 5: Implementing NOT Gate Using a NAND Gate

Theory:

1. A NOT gate can be implemented using **one NAND gate**.
2. Both inputs of the NAND gate are connected to the same input **A**.
3. The NAND gate performs the operation $(A \cdot A)'$.
4. Since $A \cdot A = A$, the output simplifies to A' .
5. Therefore, a single NAND gate acts as a **NOT gate**.

Circuit Diagram:



Truth Table:

A	$Y = (A \cdot A)' = A'$
0	1
1	0

Discussion

A NOT gate is implemented by connecting both inputs of a NAND gate to the same input signal A. The NAND gate first performs the AND operation on the identical inputs and then inverts the result. Since $A \cdot A = A$, the output becomes $(A \cdot A)' = A'$. Thus, the circuit functions exactly like a standard NOT gate. This implementation demonstrates the versatility of the NAND gate as a universal logic gate.

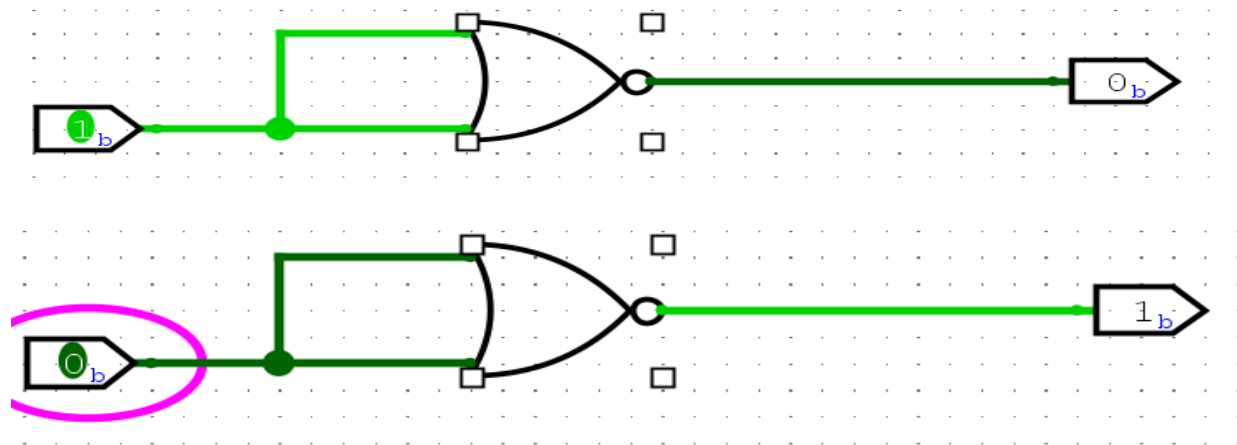
Task 6: Implementing NOT Gate Using a NOR Gate

Theory:

1. A NOT gate can be implemented using **one NOR gate**.
2. Both inputs of the NOR gate are connected to the same input A.
3. The NOR gate performs the operation $(A + A)'$.
4. Since $A + A = A$, the output simplifies to A' .

5. Therefore, a single NOR gate acts as a **NOT gate**.

Circuit Diagram:



Discussion

A NOT gate is implemented by connecting both inputs of a NOR gate to the same input signal **A**. The NOR gate first performs the OR operation on the identical inputs and then inverts the result. Since $A + A = A$, the output becomes $(A + A)' = A'$. Thus, the circuit functions exactly like a standard NOT gate. This implementation demonstrates that the **NOR gate** is a universal logic gate capable of performing the NOT operation.

Task 7: Implementing Full Adder Using Universal Gates

Theory:

1. A **Full Adder** can be implemented using only **NAND gates** or only **NOR gates**, since both are universal gates.
2. A full adder has three inputs (**A**, **B**, **Cin**) and two outputs (**Sum** and **Carry Out (Cout)**).
3. The **Sum** output is obtained using the expression $\text{Sum} = A \oplus B \oplus \text{Cin}$, while the **Carry** output is $\text{Cout} = AB + AC_{\text{in}} + BC_{\text{in}}$.
4. By replacing all AND, OR, and XOR operations with equivalent NAND-only or NOR-only circuits, the complete full adder can be realized using only one type of universal gate.
5. This implementation demonstrates that universal gates are sufficient to construct any digital logic circuit, including a full adder

Circuit Diagram:

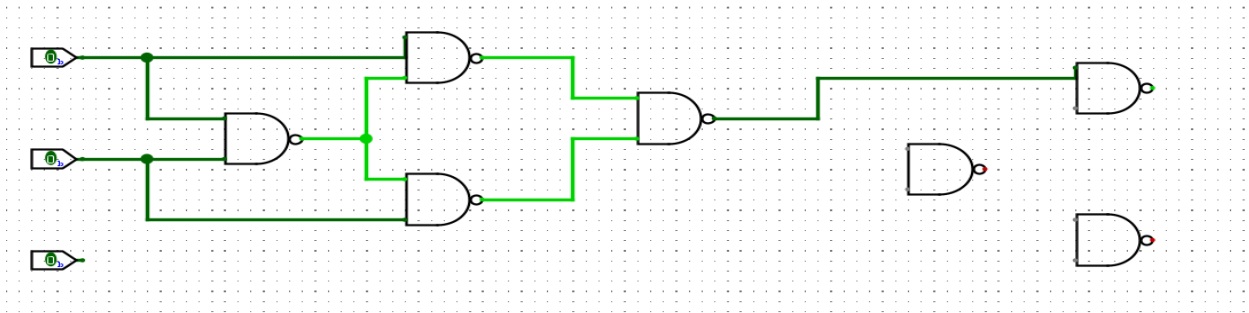


Figure 1:under construction

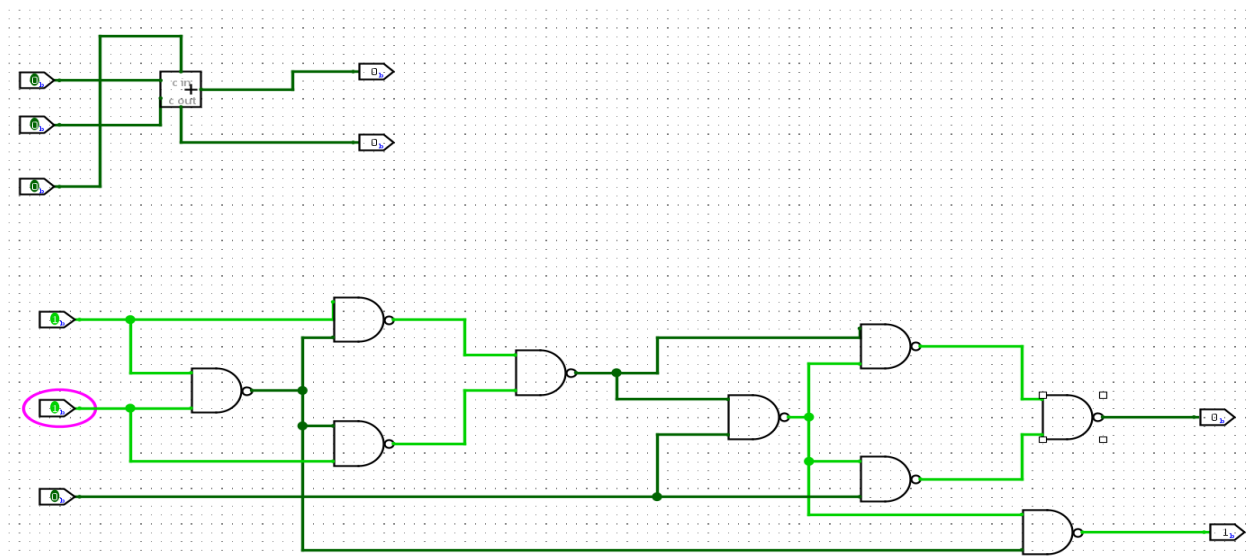


Figure 2 Full Adder using NAND gate

Truth Table:

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1

A	B	Cin	Sum	Cout
1	1	1	1	1

Discussion

A full adder is a combinational logic circuit that adds three one-bit binary inputs: **A**, **B**, and **Cin** (carry input). It produces two outputs: **Sum** and **Carry Out (Cout)**. When implemented using universal gates, all required logic functions such as AND, OR, and XOR are replaced with equivalent **NAND-only** or **NOR-only** networks. Although the circuit requires more gates than the conventional implementation, it performs the same binary addition correctly. This implementation proves that **NAND** and **NOR** gates are universal gates capable of realizing any Boolean function.

Task 8: Binary to BCD Conversion

Theory

1. A **Binary to BCD (Binary-Coded Decimal) Converter** converts a binary number into its equivalent decimal digits represented in BCD format.
2. In BCD, each decimal digit is represented by a separate **4-bit binary code** (0000–1001).
3. The converter accepts a binary input and generates separate BCD outputs for the **hundreds, tens, and units** digits.
4. Binary-to-BCD conversion is commonly performed using the **Double Dabble (Shift-and-Add-3)** algorithm or dedicated combinational logic.
5. This conversion is widely used for driving **7-segment displays**, digital clocks, calculators, and other digital display systems.

Circuit Diagram:

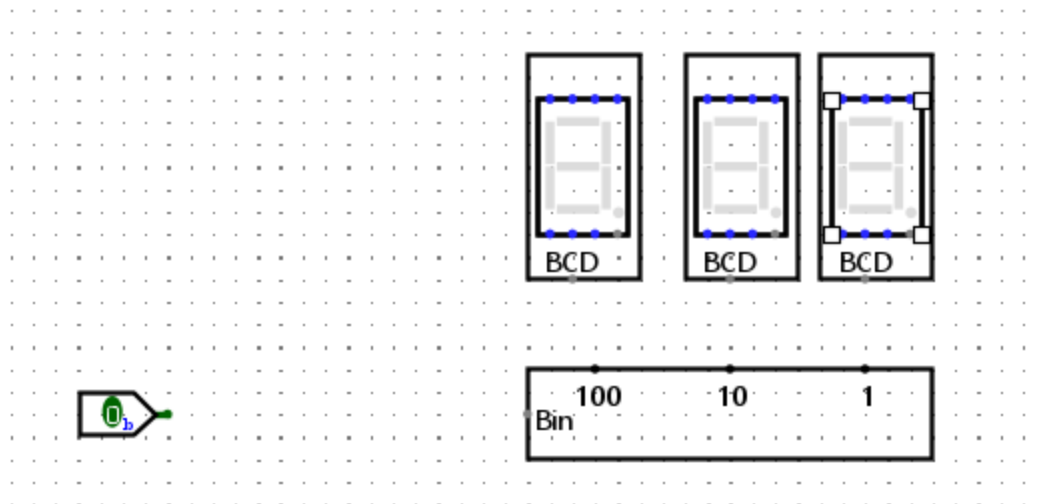


Figure 3: Under Construction

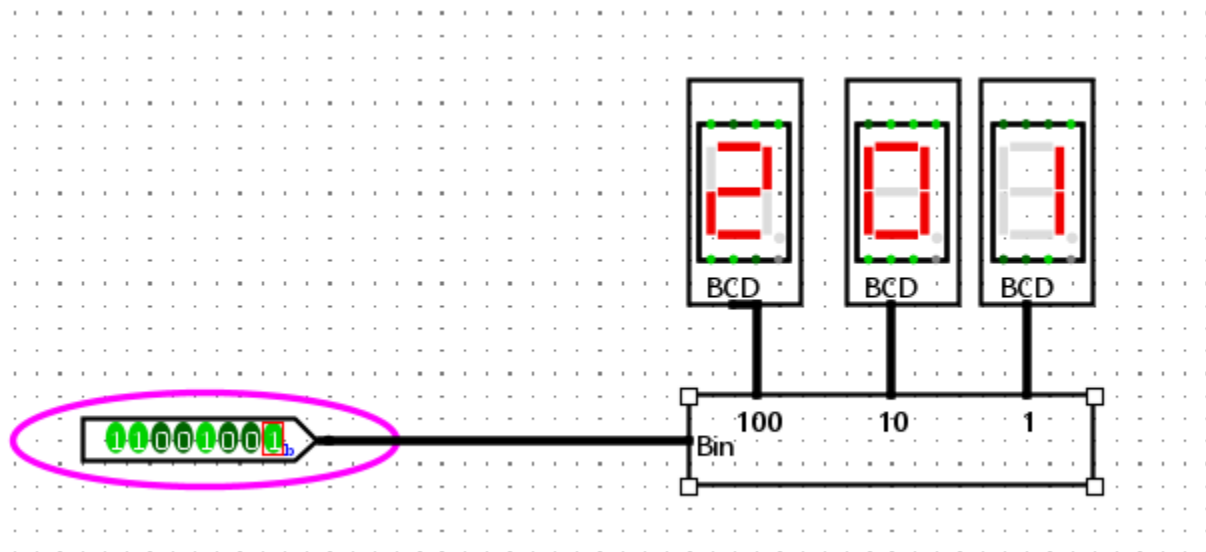


Figure 4: Binary to BCD

Discussion

The circuit shown converts an **8-bit binary input** into its equivalent **BCD representation**. The binary value is applied to the converter, which separates the decimal value into **hundreds, tens, and units**. Each BCD output is then connected to a BCD display module that drives a 7-segment display.

In the given circuit, the binary input is:

100100100₂

This binary number equals:

292_{10}

Therefore, the converter produces:

- **Hundreds = 2**
- **Tens = 9**
- **Units = 2**

These digits are displayed individually on the three BCD displays, confirming the correct operation of the Binary-to-BCD converter.